

University “Politehnica” of Bucharest
Faculty of Electronics, Telecommunications, and Information Technology

OOP Semester Project

(Computers Programming Course - Semester II)

Student: Vlad Gabriel-Lucian
Group: 411G

- Introduction

Let us consider that you have a fully developed character fighting strategy game and you want to add some new characters to it. This program could come in very handy when it comes to balancing the game, giving you the chance to try out different stats for your characters in order to see how many times a certain fighter is going to win compared to the others. Based on the data you collect from these simulations you can create statistics that would help you buff or nerf the characters such that the game is balanced.

- Description Of The Program In Natural Language

The program prompts the user to input the stats for both of the characters (name, health, attack damage, mana) and then the code will simulate a fight between these two characters on a turn-based conduct. The actions the characters do are random and are based on their mana. The simulation goes on until one of the characters dies (health level reaches 0). After each turn, the program will show the opponent's stats so that we can easily track how the simulation unfolds (this is where we can collect data for statistics).

There two types of fighters in this program: the Barbarian and the Mage. Each one of them is different and don't have common actions. The Barbarian default action (that doesn't require any mana) is the Sword Attack, but there is a special actions that costs 5 mana points, the Rage Attack, if this actions happens the Barbarian will deal three times the Sword Attack damage. The Mage default action is healing, it will heal by 3% its current health level. The special action for the Mage, which costs 4 mana points is the Fireball which deals a certain amount of damage (inputted by the user). For both of the characters, if they perform the default action, there will also be given +1 mana point.

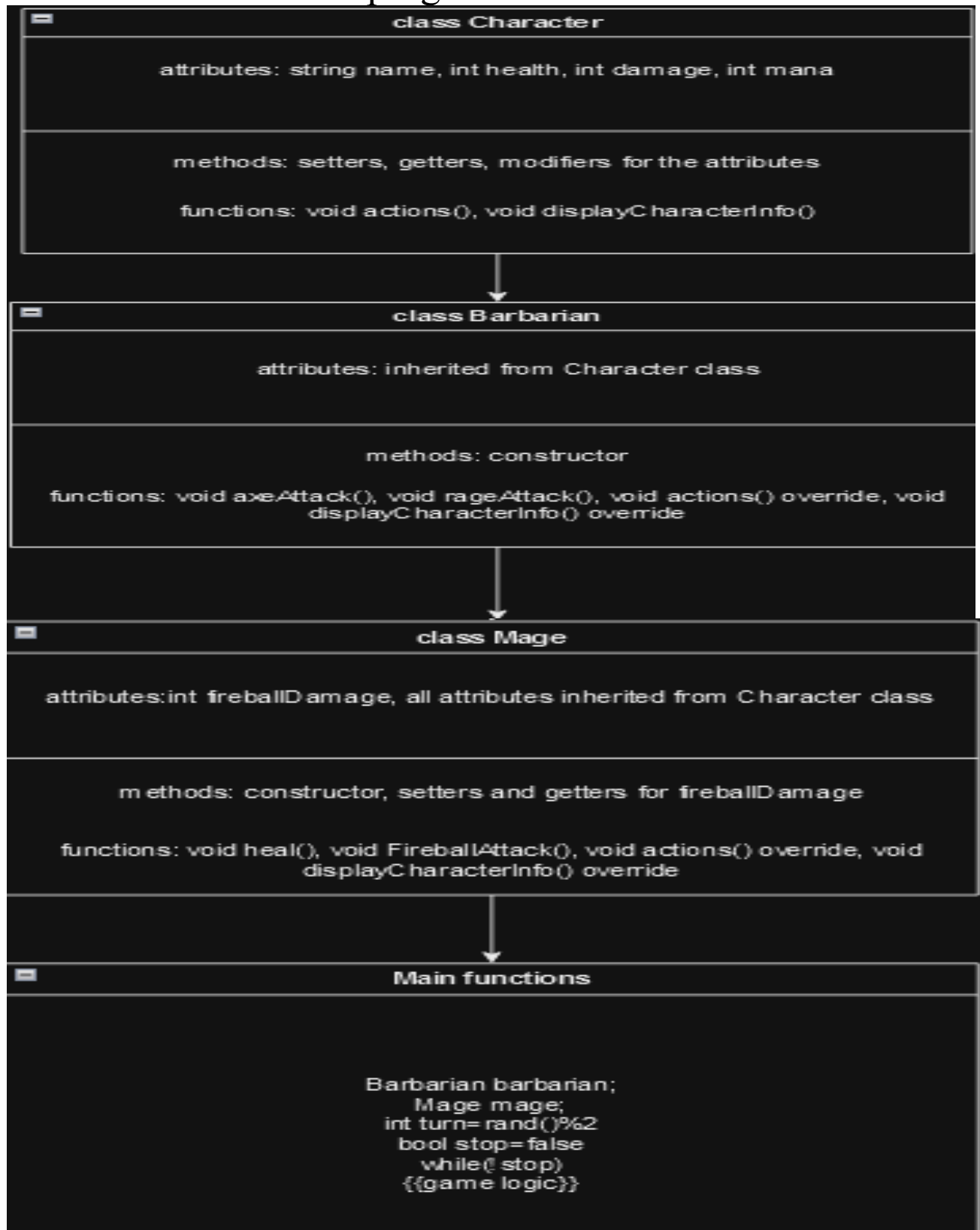
● Program Structure

In order to build this program three main classes were used: Character, Barbarian, Mage. The Barbarian class is derived from the Character class, therefore inheriting all of its components. The same can be said about the Mage class, except that an extra fireballDamage component was added along with the inherited components from the Character class.

1. The base class **Character** hold basic information about the fighters
 - a. This class contains the following protected attributes:
 - i. string name: contains the user-inputted name of the fighter
 - ii. int health: contains info about the user-inputted health points
 - iii. int damage: contains info about user-inputted damage points
 - iv. int mana: contains info about user-inputted mana points
 - b. This class contains the following public methods:
 - i. setters and getters for the attributes
 - ii. modifiers for each attribute (“modifyName()”;
”modifyHealth()”; ”modifyDamage()”; “modifyMana()”
 - iii. other methods(functions)
 - virtual void actions(): defines the behavior of the character
 - virtual void displayCharacterInfo(): displays character info

2. The class **Barbarian** which inherits from the Character class
 - a. This class contains the same attributes from the base class
 - b. This class contains the following public methods:
 - i. constructor to initialize the inherited attributes
 - ii. other methods(functions)
 - void axeAttack(): describes the logic of the default attack
 - void rageAttack(): describes the logic of the special attack
 - void actions() override: defines the behavior of the character interacting with the other fighter
 - void displayCharacterInfo() override: displays the modified character info
3. The class **Mage** which inherits from the Character class
 - a. This class contains the following protected attributes:
 - i. int fireballDamage: contains the user-inputted fireball damage points
 - ii. all of the attributes contained in the base Character class
 - b. This class contains the following methods:
 - i. constructor to initialize the attributes
 - ii. setter and getters for “fireballDamage”
 - iii. other methods(functions)
 - void heal(): describes the default ability of the mage: healing
 - void FireballAttack(): describes how the special ability works
 - void actions() override: defines the behavior of the character interacting with the other fighter
 - void displayCharacterInfo() override: displays the modified character info
4. The main() function:
 - a. create instances of “Barbarian” and “Mage”
 - b. runs the turn-based game logic and checks for winning conditions

- Flowchart for the program



- The Listing For Entire Program

```
1  #include <iostream>
2  #include <string>
3  #include <cstdlib>
4
5  using namespace std;
6
7  class Character{
8  protected:
9      string name;
10     int health;
11     int damage;
12     int mana;
13
14 public:
15
16     virtual void displayCharacterInfo(){};
17     virtual void actions(Character &c){};
18
19     void setName(string u_name) {
20         this->name = u_name;
21     }
22     void setHealth(int u_health) {
23         this->health = u_health;
24     }
25     void setDamage(int u_damage) {
26         this->damage = u_damage;
27     }
28     void setMana(int u_mana) {
29         this->mana = u_mana;
30     }
31
32     string getName() {
33         return name;
34     }
35     int getHealth() {
36         return health;
37     }
38     int getDamage() {
39         return damage;
40     }
41     int getMana() {
42         return mana;
43     }
44
45     void modifyName(){
46         string n;
47         cout<<"Enter character's name: ";
48         cin>>n;
49         this->setName( u_name: n);
50     }
51
52     void modifyHealth(){
53         int h;
54         cout<<"Enter character's health: ";
55         cin>>h;
56         this->setHealth( u_health: h);
57     }
58
59     void modifyDamage(){
60         int d;
61         cout<<"Enter character's damage: ";
62         cin>>d;
```

```

59     this->setDamage( u_damage: d);
60 }
61 void modifyMana(){
62     int m;
63     cout<<"Enter character's mana: ";
64     cin>>m;
65     this->setMana( u_mana: m);
66 }
67 };
68
69 ⑥ class Barbarian: public Character{
70 public:
71     Barbarian(){
72         cout<<"Create the Barbarian:"<<endl;
73         this->modifyName();
74         this->modifyDamage();
75         this->modifyHealth();
76         this->modifyMana();
77     }
78 void axeAttack(Character &c){
79     c.setHealth( u_health: c.getHealth() - this->getDamage());
80     this->setMana( u_mana: this->getMana()+1);
81 }
82 void rageAttack(Character &c){
83     c.setHealth( u_health: c.getHealth()- (3*getDamage()));
84     this->setMana( u_mana: this->getMana()-5);
85 }
86 ⑥ void actions(Character &c) override{
87     int attack = rand()%2;
88     if(attack==0) {
89         cout << this->getName() << ": Used the axe to attack" << endl;
90         this->axeAttack( &c);
91     }else{
92         if(this->getMana()>=5){
93             cout<<this->getName()<<": Used the rage attack"<<endl;
94         }else{
95             cout << this->getName() << ": Used the axe to attack" << endl;
96             this->axeAttack( &c);
97         }
98     }
99 }
100 ⑥ void displayCharacterInfo() override{
101     cout<<endl<<"=====Barbarian======"<<endl;
102     cout<<"Name: "<<this->getName()<<endl;
103     cout<<"Health: "<<this->getHealth()<<endl;
104     cout<<"Axe Damage: "<<this->getDamage()<<endl;
105     cout<<"Mana: "<<this->getMana()<<endl;
106     cout<<endl<<"===== "<<endl;
107 }
108 };
109
110 ⑥ class Mage:public Character{
111 public:
112     int fireballDamage;
113
114     Mage(){
115         cout<<"Create the Mage: "<<endl;
116         this->modifyName();
117         this->modifyHealth();

```

```

118         this->modifyMana();
119         //this->modifyDamage();
120         this->modifyFireballDamage();
121     }
122     void setFireballDamage(int fd){
123         this->fireballDamage=fd;
124     }
125     int getFireballDamage(){
126         return this->fireballDamage;
127     }
128     void modifyFireballDamage(){
129         int fd;
130         cout<<"Enter the fireball damage: ";
131         cin>>fd;
132         this->setFireballDamage(fd);
133     }
134
135     void heal(){
136         this->setHealth( u_health: this->getHealth() * 3/100 + this->getHealth());
137         this->setMana( u_mana: this->getMana()+1);
138     }
139     void FireballAttack(Character &c){
140         c.setHealth( u_health: c.getHealth()-this->getFireballDamage());
141         this->setMana( u_mana: this->getMana()-4);
142     }
143     void actions(Character &c) override{
144         int attack=rand()%2;
145         if(attack==0){
146             cout<<this->getName()<<": Used the healing ability."<<endl;
147             this->heal();
148         }
149         else{
150             if(this->getMana()>=4){
151                 cout<<this->getName()<<": Casted a fireball."<<endl;
152                 this->FireballAttack( &c);
153             }else{
154                 cout<<this->getName()<<": Used the healing ability."<<endl;
155                 this->heal();
156             }
157         }
158     }
159
160     void displayCharacterInfo() override{
161         cout<<endl<<"=====Mage======"<<endl;
162         cout<<"Name: "<<this->getName()<<endl;
163         cout<<"Health: "<<this->getHealth()<<endl;
164         cout<<"Fireball Damage: "<<this->getFireballDamage()<<endl;
165         cout<<"Mana: "<<this->getMana()<<endl;
166         cout<<endl<<"======"<<endl;
167     }
168
169 };
170
171 int main() {
172     Barbarian barbarian;
173     Mage mage;
174
175     int turn = rand() % 2;
176
177     bool Stop = false;

```

```

178 while(!Stop){
179
180     if(barbarian.getHealth() <= 0){
181         cout<<barbarian.getName()<<" is dead! "<<mage.getName()<<" won! Thank you for playing"<<endl;
182         Stop = true;
183     }
184
185     if(mage.getHealth() <= 0){
186         cout<<mage.getName()<<" is dead! "<<barbarian.getName()<<" won! Thank you for playing"<<endl;
187         Stop = true;
188     }
189
190     if(mage.getHealth() > 0 && barbarian.getHealth() > 0) {
191         if(turn == 0) {
192             barbarian.actions( &mage);
193             mage.displayCharacterInfo();
194         }else{
195             mage.actions( &barbarian);
196             barbarian.displayCharacterInfo();
197         }
198         turn = !turn;
199     }
200 }
201
202 return 0;
203 }

```

[click to access](#)

SOURCE CODE

- Instances of Running The Program

```
=====Mage=====
Name: Mage
Health: 258
Fireball Damage: 50
Mana: 37

=====
Mage: Used the healing ability.

=====Barbarian=====
Name: Barbarian
Health: 470
Axe Damage: 15
Mana: 16

=====
Barbarian: Used the axe to attack

=====Mage=====
Name: Mage
Health: 250
Fireball Damage: 50
Mana: 38

=====
Mage: Used the healing ability.

=====Barbarian=====
Name: Barbarian
Health: 470
Axe Damage: 15
Mana: 17

=====
Barbarian: Used the axe to attack

Create the Barbarian:
Enter character's name: Barbarian
Enter character's damage: 15
Enter character's health: 520
Enter character's mana: 20
Create the Mage:
Enter character's name: Mage
Enter character's health: 310
Enter character's mana: 40
Enter the fireball damage: 50
Mage: Casted a fireball.

=====Barbarian=====
Name: Barbarian
Health: 470
Axe Damage: 15
Mana: 20

=====
Barbarian: Used the axe to attack

=====Mage=====
Name: Mage
Health: 295
Fireball Damage: 50
Mana: 36

=====
Mage: Used the healing ability.

=====Barbarian=====
Name: Barbarian
Health: 470
Axe Damage: 15
Mana: 21

=====
Barbarian: Used the rage attack
```

```
=====Mage=====
Name: Mage
Health: 242
Fireball Damage: 50
Mana: 39

=====
Mage: Used the healing ability.

=====Barbarian=====
Name: Barbarian
Health: 470
Axe Damage: 15
Mana: 18

=====
Barbarian: Used the rage attack

=====Mage=====
Name: Mage
Health: 204
Fireball Damage: 50
Mana: 40

=====
Mage: Casted a fireball.

=====Barbarian=====
Name: Barbarian
Health: 420
Axe Damage: 15
Mana: 13

=====
Barbarian: Used the rage attack
Barbarian: Used the rage attack

=====Mage=====
Name: Mage
Health: 43
Fireball Damage: 50
Mana: 30

=====
Mage: Used the healing ability.

=====Barbarian=====
Name: Barbarian
Health: 320
Axe Damage: 15
Mana: 0

=====
Barbarian: Used the axe to attack

=====Mage=====
Name: Mage
Health: 29
Fireball Damage: 50
Mana: 31

=====
Mage: Casted a fireball.

=====Barbarian=====
Name: Barbarian
Health: 270
Axe Damage: 15
Mana: 1

=====
```

```

=====
Barbarian: Used the axe to attack

=====Mage=====
Name: Mage
Health: 14
Fireball Damage: 50
Mana: 27

=====
Mage: Used the healing ability.

=====Barbarian=====
Name: Barbarian
Health: 270
Axe Damage: 15
Mana: 2

=====
Barbarian: Used the axe to attack

=====Mage=====
Name: Mage
Health: -1
Fireball Damage: 50
Mana: 28

=====
Mage is dead! Barbarian won! Thank you for playing

Process returned 0 (0x0)   execution time : 28.708 s
Press any key to continue.
|

```

• Conclusion

Wrapping everything up we could say this program could be a very good prototype for a pre-implementation phase of a game, precisely testing characters' behavior when interacting with others. Using the base Character class we can add as many characters as we wish, providing it with a very high level of scalability. Also the main function is quite simple, very easy to understand and we can add changes and improvements to the simulation game whenever we want.

In order to build this program we used three different classes, two of them inheriting from one base class. The attributes inside the base class are very general and we can add more inheritors classes to it.

Bibliography

1. <https://www.geeksforgeeks.org/function-overriding-in-cpp/>
-used this source for implementing the overriding function
2. All PCLP 2 laboratory platforms provided on Moodle during this semester

The source code for my program can be found here:

https://github.com/VladGabrielLucian/Semester-II-OOP-Project/blob/main/source_code